

Battery-Constrained PDDL+ Rover Planning: Effects of Memory, Workload, and Data Degradation

Endri Gjinaj
Robotics Engineering
University of Genoa
Genoa, Italy

Abstract—A planetary rover must coordinate scientific work with limited memory, finite energy, travel time, and data that becomes unusable if it is not returned to base soon enough. This paper studies those interactions in a reproducible PDDL+ model. The rover crosses seeded terrain, collects and encodes datasets, stores them in onboard memory, and offloads them at a base station. Movement consumes terrain-dependent energy from a fixed 40-unit battery, while a deterministic corruption state continues to increase after collection. The final experiment crosses dataset count, memory level, and degradation margin, with five unseen seeded replicates per cell, producing 135 missions. ENHSP with `sat-hadd` was run with a standard 30-second limit; three timeouts were rerun only to obtain a final feasibility label. Success fell from 100% with two datasets to 60% with four, while it increased from 46.7% at low memory to 100% at high memory. Memory became decisive only as workload grew. All three tested degradation margins produced 80% success, but their matched runtime distributions differed: wider margins created a heavier search-time tail. Within the selected range, memory and battery determined reachability while temporal slack still affected search. Runtime increased sharply and the battery margin narrowed as workload grew.

Index Terms—PDDL+, automated planning, planetary rover, resource constraints, data degradation, ENHSP, ROS 2, reproducible experiments

I. INTRODUCTION

Planetary-rover missions are difficult to plan because their resources are coupled. A science target may be reachable and its dataset may fit in memory, yet the complete mission can still fail after travel time, battery use, storage pressure, and data-validity limits are considered together. Operational tools such as MAPGEN showed that automated planning can support rover teams in building schedules that satisfy temporal and engineering constraints [1], [2]. Later work continued to study consumptive resources and flexible execution in planetary missions [3], [4].

PDDL2.1 introduced durative actions and numeric fluents for temporal planning [5]. PDDL+ added autonomous processes and events, which makes it suitable for deterministic mixed discrete-continuous systems [6]. In the model used here, an action starts movement, a process advances the rover, and an event completes arrival. Encoding and data corruption continue while the rover is moving. Route and storage decisions therefore affect whether a collected dataset can still be returned safely.

The study asks: *How do onboard memory, mission size, and time-dependent data degradation affect feasibility, generated-*

plan efficiency, and planning difficulty in a battery-constrained rover mission?

The question does not assume that every factor must produce a significant main effect. The aim is to identify which constraints become bottlenecks under the tested conditions and how their interactions change the planning problem. The contribution is a complete experimental workflow: a PDDL+ rover domain, deterministic problem generator, ENHSP runner and parser, frozen factorial experiment, statistical notebook, and ROS 2 service/client check.

II. RELATED WORK

Automated planning has long supported space operations. MAPGEN combined planning, scheduling, and human decision making for the Mars Exploration Rover mission [1]. Bresina *et al.* describe how temporal constraints and oversubscribed resources were handled during daily rover activity planning [2]. These systems establish the operational importance of resource-aware planning, although they were not designed as controlled sensitivity studies.

Hybrid planning has also been applied to autonomous planetary vehicles. Della Penna *et al.* considered energy, time, movement, and safety in a resource-optimal rover model [7]. PDDL+ provides the formal basis for such domains by combining actions, continuous processes, and exogenous events [6]. Planning methods must then manage the search difficulty introduced by numeric evolution and event timing [8], [9]. ENHSP uses heuristic methods for numeric planning, including interval-based relaxations [10]. These works motivate treating runtime as an experimental outcome rather than recording only whether a plan was found.

The present work studies a narrower question: how memory, science workload, and deterministic degradation margins interact under a fixed battery budget. Uncertain execution and stochastic failure are important in rover operations [4], but the current PDDL+ model is deterministic. A probabilistic extension would require a stochastic formalism such as RDDDL [11], or a scenario-based approximation.

III. ROVER MODEL AND SOFTWARE PIPELINE

A. Mission Model

The map is a bidirectional line containing a base and one science site for each dataset. Every edge receives one of four terrain profiles: easy, moderate, rocky, or steep. Their travel

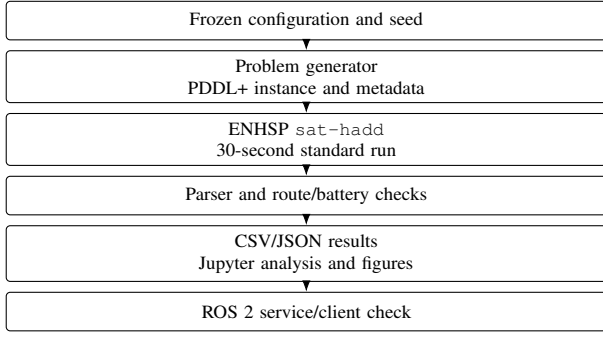


Fig. 1. Experiment and software-integration pipeline.

times are 1.5, 2.0, 2.5, and 3.0 time units, with movement-energy costs of 2, 3, 4, and 5 units. Terrain is generated deterministically from the mission seed.

Movement is represented by a `start-move` action, a continuous `rover-travel` process, and an automatic `arrive` event. The action checks that sufficient battery remains and subtracts the edge cost. The process advances travel progress until the arrival event updates the rover position.

Each dataset has a size, encoding duration, corruption value, and loss threshold. Collection occupies memory and starts both encoding and corruption. Encoding completion and data loss are automatic events. A dataset may be offloaded only at the base after encoding is complete; offloading releases its memory. The goal requires every dataset to be offloaded before it is lost.

The terms *corruption* and *degradation* describe the same mechanism at two levels. The implementation uses a numeric corruption state that increases linearly. The paper uses degradation margin to describe the positive time slack between the earliest direct offload and the loss threshold. Thus, changing the margin changes the deadline, not the corruption rate. Corruption remains active in every mission, even when the tested margins produce the same solved count.

For dataset sizes s_i , memory capacity is

$$M = \max \left(\max_i s_i, \left\lceil r \sum_i s_i \right\rceil \right), \quad (1)$$

where r is 0.45, 0.70, or 1.00 for low, medium, or high memory. The first term guarantees that every dataset fits individually. The loss time for dataset i is

$$t_i^{\text{loss}} = \max(t_i^{\text{enc}}, t_i^{\text{return}}) + \delta_i, \quad (2)$$

where δ_i is the sampled degradation margin. Since every δ_i is positive, a dataset is individually recoverable by a direct return. A complete instance can still fail through repeated trips, overlapping deadlines, memory pressure, and battery use. Battery capacity is fixed at 40 units in every instance.

B. Implementation

Figure 1 shows the software path. A frozen JSON configuration and seed are passed to a Python generator, which creates a PDDL+ problem and matching metadata. ENHSP is executed

TABLE I
FINAL EXPERIMENT SETTINGS

Component	Values
Dataset count	2, 3, 4
Memory ratio	0.45, 0.70, 1.00
Degradation margin	wide, medium, tight
Margin values	10–14, 6–10, 3–5
Final seeds	10–14
Replicates per cell	5
Terrain travel time	1.5, 2.0, 2.5, 3.0
Terrain energy cost	2, 3, 4, 5
Battery capacity	40
Planner	ENHSP <code>sat-hadd</code>
Standard limit	30 s
Instances	135

with `sat-hadd`; the parser records planner outcome, runtime, actions, makespan, and temporal information. A validation step reconstructs the route, movement energy, and remaining battery. Results are stored in CSV and JSON form and analysed in Jupyter. The same generator and planner runner are exposed through a ROS 2 service.

IV. EXPERIMENTAL METHODOLOGY

A. Design

The final study uses a balanced $3 \times 3 \times 3$ design with five seeded replicates per cell. The manipulated factors are dataset count, memory level, and degradation-margin condition; seed supplies repeatable mission variation rather than a fourth experimental factor. Table I lists the settings. Dataset sizes range from 2 to 6 units and encoding durations from 2 to 5 time units. The wide, medium, and tight margin conditions use positive slack values of 10–14, 6–10, and 3–5 time units, respectively. These values deliberately avoid trivial single-dataset failures. The factor measures whether the complete planner can absorb different amounts of temporal slack while also handling storage, travel, and battery limits.

Seeds 0–4 were used while checking and calibrating the model. The configuration was frozen before seeds 10–14 were generated. Separate deterministic random streams preserve fair comparisons: changing memory or degradation margin does not silently regenerate terrain or dataset properties.

B. Outcomes and Analysis

The main feasibility outcome is the final solved or unsolvable classification. Runtime uses only the original 30-second runs. Three timeouts were rerun with a 120-second limit to obtain a final classification, but those longer runtimes were not mixed into the fixed-budget runtime analysis.

Secondary measurements include makespan, movement actions, energy used, and battery remaining. Plan metrics are analysed for the final solved set. Comparisons across memory levels use complete matched conditions in which all three memory variants were solved. Because missions are matched across factor levels by their other settings and seed, Cochran's Q is used for binary feasibility and Friedman tests with Kendall's W for runtime and plan metrics. Exact McNemar

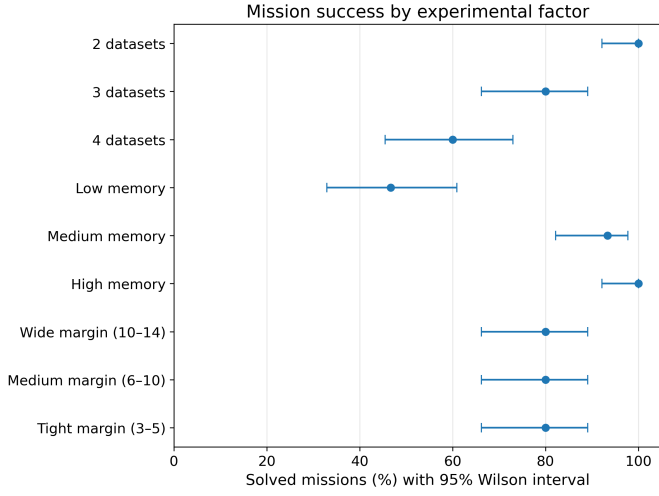


Fig. 2. Mission success by factor. Each point contains 45 missions; error bars are 95% Wilson intervals.

and Wilcoxon signed-rank follow-ups use Holm adjustment. Wilson intervals accompany success percentages. With five seeded replicates per cell, inference is treated as exploratory rather than as a universal estimate of rover performance.

V. RESULTS

A. Feasibility

The standard runs produced 107 solved instances, 25 unsolvable instances, and three timeouts. Extended checks classified two timeouts as unsolvable and one as solved. The final result was therefore 108 solved missions and 27 unsolvable missions, with no unresolved cases.

Figure 2 summarizes success by factor. Success fell from 100% for two datasets to 80% for three and 60% for four. It increased from 46.7% at low memory to 93.3% at medium memory and 100% at high memory. Wide, medium, and tight degradation margins all produced 80% success.

The interaction in Fig. 3 is more informative than the marginal values alone. Memory had no visible effect on two-dataset missions because every case was solved. With three datasets, low memory solved 40% of missions, while medium and high memory solved all of them. With four datasets, low memory solved none, medium memory solved 80%, and high memory solved every mission.

Matched feasibility changed across dataset counts, $Q(2) = 27.00$, $p = 1.37 \times 10^{-6}$, and memory levels, $Q(2) = 42.75$, $p = 5.21 \times 10^{-10}$. Holm-adjusted exact McNemar tests separated all three workload levels. Low memory differed from medium and high memory, while the 93.3% and 100% rates for medium and high memory were not detectably different ($p_{\text{adj}} = 0.25$). All 45 matched triplets had identical feasibility across degradation margins, giving $Q(2) = 0$, $p = 1$. This identifies equal reachability in the selected range; it does not remove corruption deadlines from the plans.

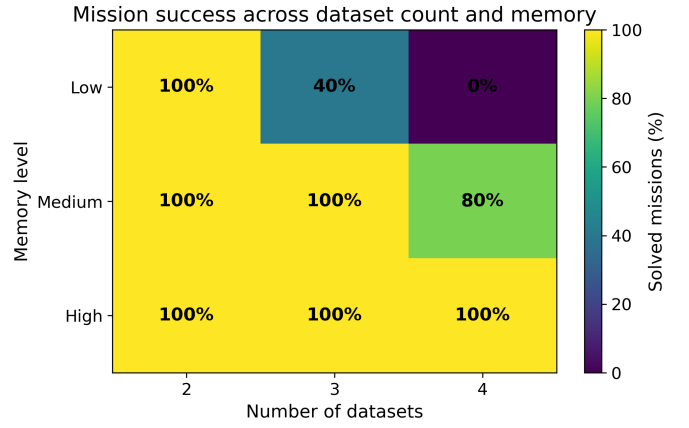


Fig. 3. Workload-memory interaction. Each cell contains 15 missions.

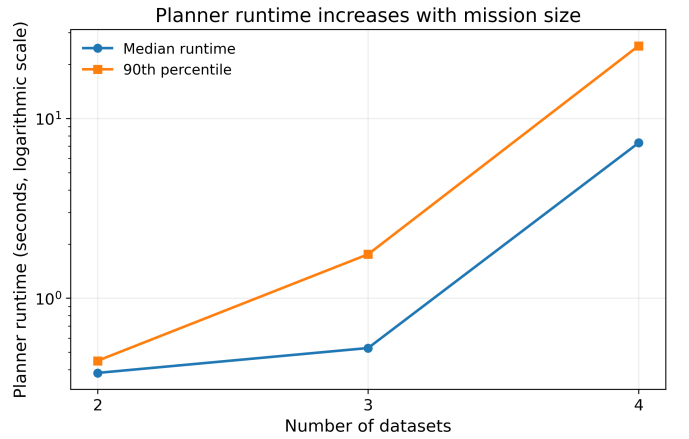


Fig. 4. Median and 90th-percentile runtime from the original 30-second runs.

B. Runtime and Battery Margin

Figure 4 compares the median and 90th percentile of standard runtime. The median rose from 0.384 s for two datasets to 0.528 s for three and 7.315 s for four. The 90th percentile rose from 0.449 s to 1.755 s and then 25.345 s. All three standard timeouts occurred in four-dataset cases. The matched dataset-count effect was large, $\chi^2_F(2) = 86.18$, $p = 1.94 \times 10^{-19}$, with $W = 0.958$.

Mean runtime decreased from 6.253 s for wide degradation margins to 3.286 s for medium margins and 1.014 s for tight margins. Figure 5 shows that the difference is mainly in the upper tail. The paired comparison was significant, $\chi^2_F(2) = 36.04$, $p = 1.49 \times 10^{-8}$, $W = 0.400$, and all three Holm-adjusted Wilcoxon follow-ups were significant ($p_{\text{adj}} \leq 2.10 \times 10^{-5}$). The margins changed search effort even though they did not change final feasibility.

Figure 6 uses the final 108 solved plans. Two-dataset missions retained an average of 21.73 battery units. The average fell to 8.44 units for three datasets and 1.70 units for four. Successful four-dataset plans therefore operated close to the fixed 40-unit battery limit.

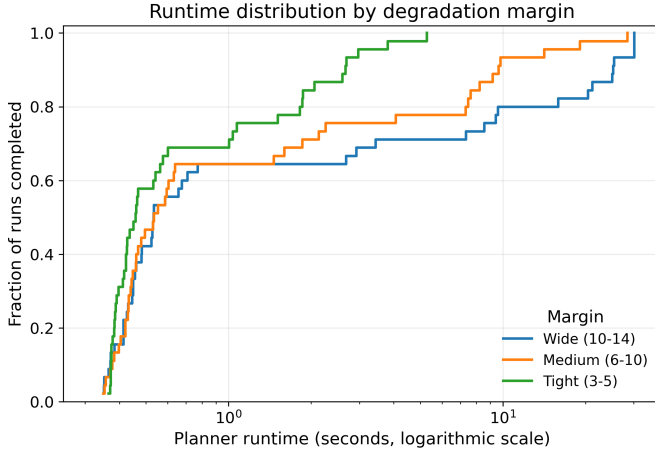


Fig. 5. Empirical runtime distributions from the matched 30-second runs. Wider degradation margins produce a heavier search-time tail.

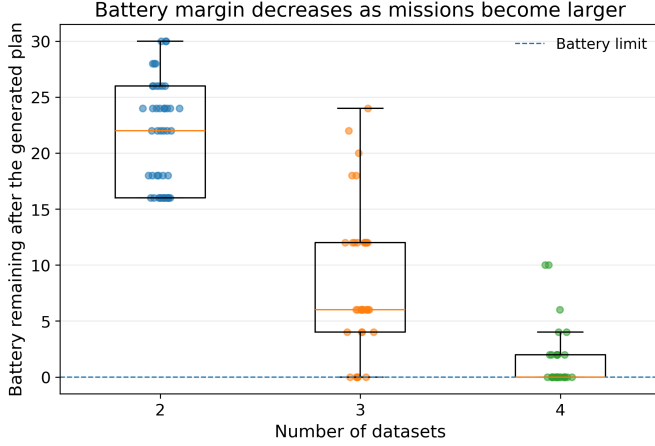


Fig. 6. Battery remaining after the final solved plans. Points show individual missions; boxes summarize each workload.

C. Generated-Plan Efficiency

For a fair memory comparison, 21 matched mission conditions were selected in which low, medium, and high memory all produced a plan. Mean movement counts were 7.71, 7.14, and 5.33; mean energy use was 24.57, 22.86, and 16.38; and mean makespan was 25.76, 23.52, and 17.10. Friedman tests found large memory effects for movement and energy, $\chi^2_F(2) = 35.29$, $p = 2.17 \times 10^{-8}$, $W = 0.840$, and for makespan, $\chi^2_F(2) = 34.24$, $p = 3.67 \times 10^{-8}$, $W = 0.815$. Every pairwise memory comparison remained significant after Holm adjustment for all three metrics; the largest adjusted value was $p_{\text{adj}} = 0.0147$. These are properties of the returned satisficing plans, not claims of global optimality.

D. ROS 2 Check

The same generator and planner runner used by the batch experiment are exposed through a ROS 2 service. In the executed notebook, a three-dataset, medium-memory, medium-margin request returned a solved plan with makespan 28, ten

movement actions, and 26 battery units used. The remaining 14 units matched the value reconstructed independently in the notebook. This confirms consistency between the ROS interface and the offline pipeline; it is not counted as an additional statistical trial.

VI. DISCUSSION

A. Resource Interaction and Bottlenecks

The clearest result is not simply that more memory is better. Memory made no difference for the smallest missions because every setting was sufficient. Its effect appeared only when workload increased. Limited storage then forced more returns to base, and those trips consumed energy while additional time passed. In the four-dataset condition, low memory solved no missions while high memory solved all of them. The experiment therefore identifies a workload-dependent storage threshold rather than an isolated memory effect.

The battery results support the same interpretation. Successful four-dataset plans retained little energy, so the largest workload operated near the battery boundary even when a plan existed. Energy and makespan rose together because both were driven by travel. These links explain why feasibility emerged from the interaction of storage, movement, timing, and energy.

B. What the Degradation Result Means

Data corruption remained active in every mission. Once a dataset was collected, the rover had a limited interval in which to finish encoding, return to base, and offload it. Removing the corruption process would remove that temporal commitment and create a different planning problem.

The equal 80% success rates mean something narrower: within the selected positive margins, moving from wide to medium to tight slack did not change the final number of solvable missions. Since terrain and dataset properties were held fixed across comparable conditions, ENHSP could usually absorb the tested changes by changing the route and offload schedule. Memory and battery reached their limits first. In this sense, the experiment identifies a region of temporal robustness; it does not show that degradation is unimportant.

The longer runtime tail under wide margins has a plausible search explanation. Wider windows admit more candidate schedules, whereas tighter windows may reject alternatives earlier. The matched runtime effect is clear in this experiment, but the pruning mechanism remains a hypothesis until it is tested with planner instrumentation or other heuristics.

VII. VALIDITY, LIMITATIONS, AND FUTURE WORK

Construct validity. The degradation variable is a deterministic clock that triggers data loss at a threshold. It captures the need to return data within a time window, but not gradual quality loss, radiation-dependent failure, uncertain storage faults, or a probability of corruption. Standard PDDL+ describes deterministic hybrid dynamics [6]. A probabilistic version would require a model such as RDDDL [11], or repeated deterministic scenarios sampled from a failure distribution.

Internal and statistical validity. Deterministic seeds, separate random streams, frozen parameters, unseen final seeds, matched tests, and explicit timeout classification reduce confounding. The main remaining limits are five seeded replicates per cell and wall-clock runtimes that include Java and planner startup. Three timeouts were selectively extended, but their longer runtimes were excluded from fixed-budget runtime analysis.

External and algorithmic validity. The linear map and synthetic terrain simplify navigation, while battery use includes movement only. Increasing dataset count also adds a more distant site, so the workload factor combines additional science tasks with greater spatial extent; this study cannot separate those two contributions. One satisficing ENHSP configuration was tested, so energy and makespan are properties of returned plans rather than global optima. The conclusions apply to this controlled domain and parameter range, not directly to a flight mission.

Future work should first sweep tighter and partially negative degradation margins to locate the reachability boundary. It should then test probabilistic failure with RDDDL or another MDP-based formalism. Nonlinear maps, solar charging, communication windows, additional energy consumers, more seeds, and planner comparisons would broaden the study.

VIII. CONCLUSION

The final experiment evaluated 135 battery-constrained rover missions while varying science workload, onboard memory, and deterministic data-degradation margins. Larger missions were less feasible, took longer to plan, and left less battery after successful execution. Memory was the strongest feasibility factor, but its importance emerged mainly as workload increased. In matched solved missions, additional memory also reduced movement, energy use, and makespan.

The tested degradation margins did not change aggregate feasibility, but they did change planner runtime: wide margins produced the heaviest search-time tail. This is not evidence that timing was irrelevant. Every collected dataset still imposed an encoding and offloading deadline. Instead, the result separates two effects: storage and battery dominated reachability in the tested range, while temporal slack influenced search difficulty.

More aggressive or probabilistic corruption remains a separate and technically meaningful extension.

The study shows why rover resources must be analysed together. Storage decisions change travel; travel changes energy and elapsed time; and those changes determine whether all data can be returned safely. The repository provides the frozen model, generator, final results, executed notebook, statistical tables, figures, and ROS 2 interface needed to reproduce and extend the analysis.

REFERENCES

- [1] M. Ai-Chang, J. Bresina, L. Charest, A. Chase, J. Hsu, A. Jonsson, B. Kanefsky, P. Morris, K. Rajan, and J. Yglesias, "MAPGEN: Mixed-initiative planning and scheduling for the Mars Exploration Rover mission," *IEEE Intelligent Systems*, vol. 19, no. 1, pp. 8–12, 2004.
- [2] J. L. Bresina, A. K. Jonsson, P. H. Morris, and K. Rajan, "Activity planning for the Mars Exploration Rovers," in *Proceedings of the 15th International Conference on Automated Planning and Scheduling*, pp. 40–49, 2005.
- [3] W. Chi, S. Chien, and J. Agrawal, "Scheduling with complex consumptive resources for a planetary rover," in *Proceedings of the Thirtieth International Conference on Automated Planning and Scheduling*, pp. 348–356, 2020.
- [4] J. Agrawal, W. Chi, S. A. Chien, G. Rabideau, D. Gaines, and S. Kuhn, "Analyzing the effectiveness of rescheduling and flexible execution methods to address uncertainty in execution duration for a planetary rover," *Robotics and Autonomous Systems*, vol. 140, p. 103758, 2021.
- [5] M. Fox and D. Long, "PDDL2.1: An extension to PDDL for expressing temporal planning domains," *Journal of Artificial Intelligence Research*, vol. 20, pp. 61–124, 2003.
- [6] M. Fox and D. Long, "Modelling mixed discrete-continuous domains for planning," *Journal of Artificial Intelligence Research*, vol. 27, pp. 235–297, 2006.
- [7] G. Della Penna, B. Intrigila, D. Magazzeni, and F. Mercorio, "Resource-optimal planning for an autonomous planetary vehicle," *International Journal of Artificial Intelligence and Applications*, vol. 1, no. 3, pp. 15–29, 2010.
- [8] A. J. Coles and A. Coles, "PDDL+ planning with events and linear processes," in *Proceedings of the 24th International Conference on Automated Planning and Scheduling*, pp. 74–82, 2014.
- [9] W. Piotrowski, M. Fox, D. Long, D. Magazzeni, and F. Mercorio, "Heuristic planning for PDDL+ domains," in *Proceedings of the 25th International Joint Conference on Artificial Intelligence*, pp. 3213–3219, 2016.
- [10] E. Scala, P. Haslum, S. Thiebaux, and M. Ramirez, "Interval-based relaxation for general numeric planning," in *Proceedings of the 22nd European Conference on Artificial Intelligence*, pp. 655–663, IOS Press, 2016.
- [11] S. Sanner, "Relational dynamic influence diagram language (RDDDL): Language description," tech. rep., Australian National University, 2011.